



Formal Verification



Aave StableVault

June 2026

Prepared for Aave Labs

Table of Contents

Project Summary.....	3
Project Scope.....	3
Project Overview.....	3
Severity Matrix.....	4
Formal Verification.....	5
Verification Methodology.....	5
Verification Notations.....	5
General Assumptions and Simplifications.....	6
Properties Overview.....	7
Detailed Properties.....	8
P01. StableVault Valid State.....	8
P02. AssetRegistryValid State.....	9
P03. FundsHandler Valid State.....	10
P04. StableVault integrity of ERC20 flow.....	10
P05. StableVault integrity of native flow.....	11
P06. StableVault integrity.....	11
P07. Allocator integrity of ERC20 flow.....	12
P08. Allocator integrity of native flow.....	12
P09. IouToken integrity.....	12
P10. MathLib equivalence with CVL.....	13
P11. AssetLib equivalence with CVL.....	14
Disclaimer.....	15
About Certora.....	15

Project Summary

Project Scope

Project Name	Repository Link	Commit Hash	Platform
Aave Stable Vault	https://github.com/aave/stable-vault	811e214	Solidity

Project Overview

This document describes the specification and verification of **AAVE V4 Stable Vault**. The work was undertaken from **May 11th, 2026** to **June 30th, 2026**.

The following contract list is included in our scope:

```
src/core/accounting/StableVault.sol
src/core/accounting/FundsHandler.sol
src/periphery/AssetRegistry.sol
src/core/Allocator.sol
src/core/ious/IouToken.sol
```

The Certora Prover demonstrated that the implementation of the above Solidity contracts are correct with respect to the formal rules written by the Certora team.

Findings Summary

No findings were discovered due to violations of properties.

Severity	Discovered	Confirmed	Fixed
Critical	-	-	-
High	-	-	-
Medium	-	-	-
Low	-	-	-
Informational	-	-	-
Total	-	-	-

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
		Likelihood		

Formal Verification

Verification Methodology

We performed verification of the **AAVE Stable Vault** protocol using the Certora verification tool which is based on Satisfiability Modulo Theories (SMT). In short, the Certora verification tool works by compiling formal specifications written in the [Certora Verification Language \(CVL\)](#) and **AAVE's** implementation source code written in Solidity.

More information about Certora's tooling can be found in the [Certora Technology Whitepaper](#).

If a property is verified with this methodology it means the specification in CVL holds for all possible inputs. However specifications must introduce assumptions to rule out situations which are impossible in realistic scenarios (e.g. to specify the valid range for an input parameter). Additionally, SMT-based verification is notoriously computationally difficult. As a result, we introduce overapproximations (replacing real computations with broader ranges of values) and underapproximations (replacing real computations with fewer values) to make verification feasible.

Rules: A rule is a verification task possibly containing assumptions, calls to the relevant functionality that is symbolically executed and assertions that are verified on any resulting states from the computation.

Inductive Invariants: Inductive invariants are proved by induction on the structure of a smart contract. We use constructors as a base case, and consider all other (relevant) externally callable functions that can change the storage as step cases.

Specifically, to prove the base case, we show that a property holds in any resulting state after a symbolic call to the respective constructor. For proving step cases, we generally assume a state where the invariant holds (induction hypothesis), symbolically execute the functionality under investigation, and prove that after this computation any resulting state satisfies the invariant.

Verification Notations

Formally Verified

The rule is verified for every state of the contract(s), under the assumptions of the scope/requirements in the rule.

General Assumptions and Simplifications

- We use a CVL-based implementation of `Math.mulDiv`, `Math.mulDivUp` and `Math.mulDivDown`. These are equivalent to the originals and better suited for verification purposes.
- Loops are inherently difficult for formal verification. We handle loops by unrolling them a specific amount of times. We use a **loop_iter of 3**, unrolling each loop up to three times.
- Underlying assets are assumed to be standard ERC20, specifically:
 - a. No fees on transfer
 - b. Non-rebasing
 - c. Without double entries
 - d. Decimals between 6 and 18
- The verified Solidity contracts are compiled with solc8.28.
- All properties are verified against the provided implementation.
- Due to the size of the code and the handling of `delegateCall`, function `multicall()` is excluded from parametric invariant induction. All functions that can be called from `multicall` are verified.

Properties Overview

ID	Title	Impact
P-01	StableVault Valid State	Direct fund loss, broken economic model
P-02	AssetRegistry Valid State	Integrity of asset tracking
P-03	FundsHandler Valid State	Integrity of earning chain
P-04	StableVault integrity of ERC20 flow	Correctness of deposit/withdrawal flows
P-05	StableVault integrity of native flow	Correctness of native flow
P-06	StableVault integrity	Integrity of core protocol functions
P-07	Allocator integrity of ERC20 flow	Consistency of balance accounting
P-08	Allocator integrity of native flow	Consistency of native balance accounting
P-09	louToken integrity	Consistency of supply and balances
P-10	MathLib equivalence with CVL	Mathematical correctness
P-11	AssetLib equivalence with CVL	Decimal conversion correctness

Detailed Properties

P01. StableVault Valid State

Status: Verified

Rule Name	Status	Description	Link to rule report
sumUserSharesEqualsSubVaultTotalShares	Verified	For each subVaultId, 'sumUserSharesBySubVault[subVaultId]' equals 'subVaultById[subVaultId].totalShares'.	rule report
globalOriginalDepositsMatchesSumOfUserOriginals	Verified	Ghost sum of 'positions[user].originalDepositRay' equals 'globalOriginalDepositsRay'.	rule report
validRateForExistingSubVault	Verified	If 'getSubVaultRateById(subVaultId) != 0', rate is at least RAY and at most max valid per-second rate.	rule report
activeSubVaultConversionRateAtLeastRay	Verified	For real subVault slots ('0 < subVaultId <= lastSubVaultId'), 'conversionRate >= RAY'. Matches on-chain init at 'MathLib.RAY' and monotonic accrual.	rule report
rateIdRoundTrip	Verified	If 'subVaultIdByRate[rate] != 0', that id's 'perSecondRate' equals 'rate' and id <= 'lastSubVaultId'.	rule report
activeSubVaultSlotRoundTrip	Verified	For 'index < length', id at slot maps back to 'index' and is a non-zero allocated id.	rule report
activeSubVaultIdRoundTrip	Verified	Non-zero index entry points at 'subVaultId'; slot 0's index entry is 0 when list non-empty.	rule report

activeSubVaultsActiveWhenHavingShares	Verified	For <code>'subVaultId > 0'</code> , <code>'totalShares > 0'</code> iff id appears at its index in <code>'activeSubVaultIds'</code> .	rule report
defaultSubVaultWellFormed	Verified	<code>'defaultSubVaultId <= lastSubVaultId'</code> once the vault has allocated subVaults.	rule report
activeSubVaultEntryWellFormed	Verified	In-range indices hold non-zero ids <code><= 'lastSubVaultId'</code> with matching index map; past length, slot is 0.	rule report
emptySubVaultHasNoShares	Verified	If <code>'subVaultId > lastSubVaultId'</code> , on-chain subVault fields, share ghost, and reverse maps are zero.	rule report
zeroSubVaultIdsEmpty	Verified	Slot 0 and any user with <code>'subVaultId == 0'</code> have zero shares and zero original deposit.	rule report
defaultSubVaultIdNonZero	Verified	Once <code>'lastSubVaultId > 0'</code> , <code>'defaultSubVaultId'</code> is non-zero and <code><= 'lastSubVaultId'</code> .	rule report
userPositionWellFormed	Verified	If <code>'positions[user].subVaultId != 0'</code> , id is allocated and <code>'shares > 0'</code> .	rule report

P02. AssetRegistryValid State

Status: Verified

Rule Name	Status	Description	Link to rule report
assetRegistryAssetsSetInvariant	Verified	Ghost mirrors of <code>'assets._inner'</code> satisfy OZ EnumerableSet index/value consistency.	rule report

assetRegistryTrustedAssetsSetInvariant	Verified	Ghost mirrors of `trustedAssets._inner` satisfy OZ EnumerableSet index/value consistency.	rule report
userDepositAllowedImpliesAssetTrusted	Verified	If an asset is enabled for user deposits, it must be trusted in the asset registry.	rule report

P03. FundsHandler Valid State

Status: Verified

Rule Name	Status	Description	Link to rule report
fundsHandlerEarningChainIdsSetInvariant	Verified	Ghost mirrors of `earningChainIds._inner` satisfy OZ EnumerableSet index/value consistency: $values[indexes[v] - 1] == v$ and $indexes[values[i]] == i + 1$.	rule report

P04. StableVault integrity of ERC20 flow

Status: Verified

Rule Name	Status	Description	Link to rule report
deposit_integrity_ERC20	Verified	On successful deposit, the depositor loses `amount` and TransferHelper net balance is unchanged (assertingTransferHelperBalanceFor + Allocator deposit summarized as NONDET).	rule report

executeWithdrawal_integrity_ERC20	Verified	On successful execution (1:1 execution policy summary), IOUs are burned and the user receives assets from TransferHelper without increasing helper net balance.	rule report
-----------------------------------	----------	---	-----------------------------

P05. StableVault integrity of native flow

Status: Verified

Rule Name	Status	Description	Link to rule report
noChangeNativeBalance	Verified	On successful or other operations, the native balance of the sender is unchanged and the allocator's balance is unchanged.	rule report

P06. StableVault integrity

Status: Verified

Rule Name	Status	Description	Link to rule report
fundsHandler_processDeposit_onlyVault	Verified	Direct calls to 'FundsHandler.processDeposit' must come from StableVault.	rule report
fundsHandler_processWithdrawal_onlyVault	Verified	Direct calls to 'FundsHandler.processWithdrawal' must come from StableVault.	rule report

P07. Allocator integrity of ERC20 flow

Status: Verified

Rule Name	Status	Description	Link to rule report
Allocator.deposit_integrity_ERC20	Verified	On successful deposit, TransferHelper balance decreases by 'amount' and Allocator balance increases by 'amount'.	rule report
withdraw_integrity_ERC20	Verified	On successful withdraw, TransferHelper balance of 'asset' increases by exactly 'amount'.	rule report

P08. Allocator integrity of native flow

Status: Verified

Rule Name	Status	Description	Link to rule report
deposit_integrity_native_not_supported	Verified	Allocator.deposit must revert when 'asset' is the native currency sentinel.	rule report
withdraw_integrity_native	Verified	On successful native withdraw, TransferHelper native balance increases by exactly 'amount'.	rule report

P09. loutoken integrity

Status: Verified

Rule Name	Status	Description	Link to rule report
totalSupplyIsSumOfBalances	Verified	<i>The ERC20 <code>_totalSupply</code> always equals <code>usum</code> of all account balances.</i>	rule report
totalSupplyChangeOnlyByOwner	Verified	<i>If any non-view call changes <code>louToken.totalSupply()</code>, <code>msg.sender</code> must be <code>louToken.owner()</code>. Matches Solidity: only <code>mint</code> and <code>burn</code> change supply and both are <code>onlyOwner</code>; <code>lock</code> and ERC20 transfers do not change supply.</i>	rule report

P10. MathLib equivalence with CVL

Status: Verified

Rule Name	Status	Description	Link to rule report
RAY_matches_common_ghost	Verified	<i>MathLib.RAY() returns the same value as the persistent RAY ghost used in summaries.</i>	rule report
rayMulDown_matches_mulDivRayDownCVL	Verified	<i>Revert conditions and return values agree on all inputs.</i>	rule report
rayMulUp_matches_mulDivRayUpCVL	Verified	<i>Revert conditions and return values agree on all inputs.</i>	rule report
rayDivDown_matches_mulDivDownCVL_with_RAY	Verified	<i>Revert conditions and return values agree on all inputs.</i>	rule report

rayDivUp_match es_mulDivUpCVL _with_RAY	Verified	<i>Revert conditions and return values agree on all inputs.</i>	rule report
---	----------	---	-----------------------------

P11. AssetLib equivalence with CVL

Status: Verified

Rule Name	Status	Description	Link to rule report
convertDecimals _matches_CVL	Verified	<i>On the modeled domain (one side's decimals in [6,18] and delta ≤ 18), whenever the CVL model produces a value, the Solidity function produces the same value and does not revert.</i>	rule report
convertDecimals _scaleUp_overflow	Verified	<i>When scaling up (inputDecimals < outputDecimals), the Solidity multiplication reverts exactly when the CVL model overflows uint256.</i>	rule report

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline and is widely used by developers and security researchers during audits and bug bounties.

Certora also provides architectural design reviews, where researchers analyze system design and trust boundaries early to identify structural risks and reduce complexity before implementation.

Certora conducts off-chain audits covering backend services, APIs, frontend, and mobile applications, focusing on vulnerabilities in authentication, data handling, key management, and wallet-related workflows.

Certora also offers Penetration Testing and Red Teaming to simulate real-world attacks and uncover exploitable weaknesses across infrastructure, applications, and business logic.

In addition, Certora provides operational security (OpSec) assessments, reviewing key management, access controls, and operational processes to strengthen overall security posture.

Finally, Certora delivers ongoing monitoring and incident response, enabling continuous threat detection and prevention, alert triage, and coordinated response to active incidents.